

— **MLOPS PIPELINE** —

PRÉDICTION DES MALADIES AUTO-IMMUNE

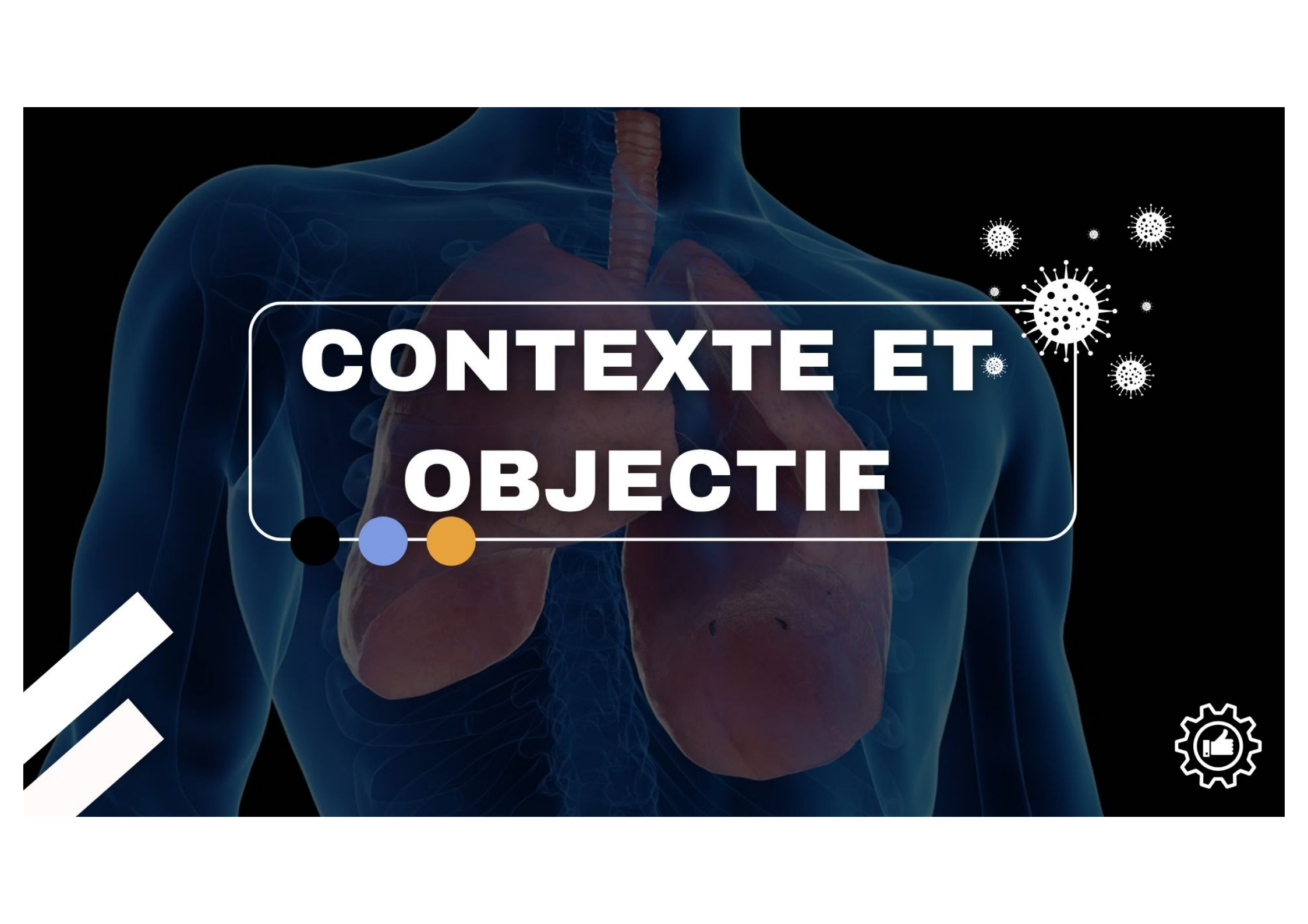


PR. MOHAMMED AIT DAOUD

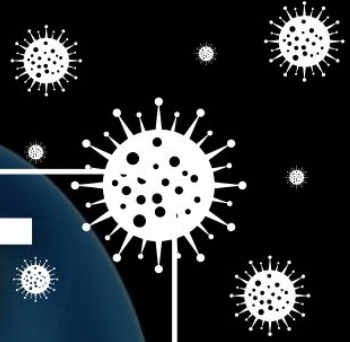
Team :

Marwa Bouzaouit
Aya Adadi
Malak Boussetta
Nora Idouaouzal

Benlaidi Hajar
Hiba Bounaga
Wahiba Bousyf
Hajar Alouani



CONTEXTE ET OBJECTIF





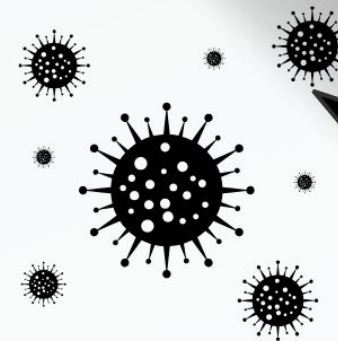
PROBLÉMATIQUE & OBJECTIF

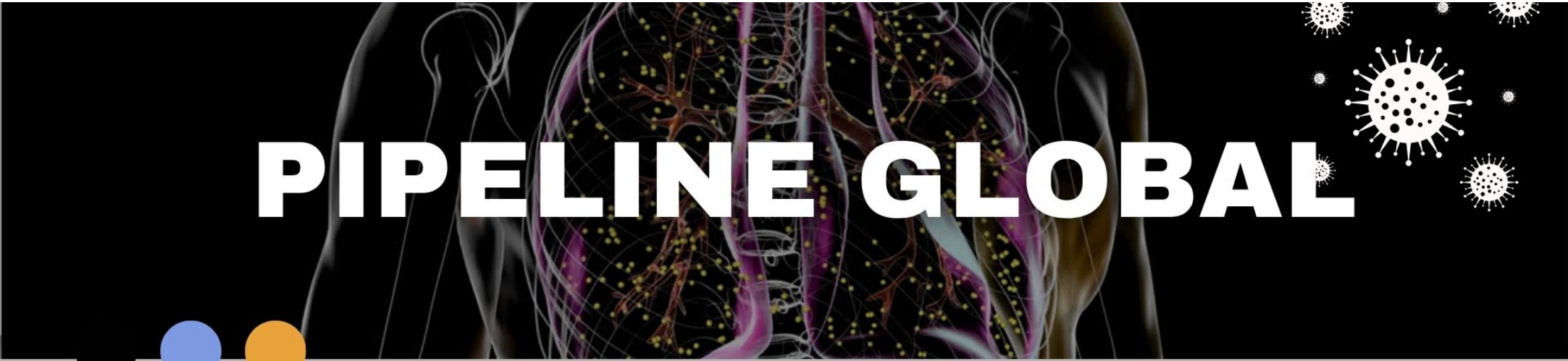
Problématique

Comment transformer un modèle de classification entraîné sur des données médicales en un système reproductible, testé, déployable et monitoré ?

Objectifs

- Automatiser l'ingestion et la transformation des données
- Assurer la qualité des données
- Entraîner et évaluer des modèles de classification
- Déployer le modèle via une API
- Conteneuriser l'application avec Docker
- Mettre en place un monitoring du système et du modèle





PIPELINE GLOBAL



01

Données &
ingestion



02

Stockage
DuckDB



03

transformations
dbt



04

Data Quality
& Data
Contracts



05

Orchestration
– Dagster



06

ML & MLflow



07

Déploiement
FastAPI +
Docker



08

CI/CD



09

Monitoring &
Observabilité



10

Résultats,
limites



ORGANISATION

Une équipe de 8 membres avec des responsabilités complémentaires

MLOps / ML	Hajar Alouani & Wahiba Bousyf
Data Engineering	Benlaidi Hajar
Data Transformation	Malak Boussetta
Data Quality	Hiba Bounaga
DevOps / CI-CD	Nora Idouaouzal
Monitoring	Aya Adadi
Product Owner & Documentation	Marwa Bouzaouit



ORGANISATION AGILE

1. Project Vision

- Définition de la problématique
- Définition des objectifs
- Définition du périmètre
- Définition des critères de réussite

2. Organisation Agile

- Construction du Product Backlog
- Rédaction des User Stories
- Organisation des tâches en sprints
- Suivi de l'avancement avec l'équipe

3. Documentation

- Documents Agile
- Suivi de la cohérence entre le backlog et le travail réalisé
- Rapport final
- Documentation du projet



SCRUM ET JIRA

Organisation du projet — Scrum & Jira

- Product Backlog → ensemble des User Stories
- **Sprint 1** – Data Engineering → ingestion, stockage, transformation, features, validation, data contracts
- **Sprint 2** – Machine Learning → entraînement, évaluation, Model Registry
- **Sprint 3** – Deployment & Monitoring → API, Docker, tests, CI, monitoring
- Jira → Planifier → Répartir → Suivre → Documenter l'avancement
- **Objectif** → assurer une organisation claire et un suivi continu du projet



DATA ENGINEER



MON RÔLE : DATA ENGINEER

Gestion & ingestion des données

- Collecte du dataset
- Préparation des données
- Mise à disposition des données pour le pipeline

Objectif : disposer de données nécessaires et et les mettre à disposition du pipeline.



PIPELINE D'INGESTION AVEC DLT

Ingestion des données

Données brutes



- Automatisation du chargement avec dlt
- Ressource : patients
- Stratégie : replace
- À chaque nouvelle ingestion, le contenu précédent de la ressource est remplacé par les données actuellement chargées.



STOCKAGE & ORCHESTRATION

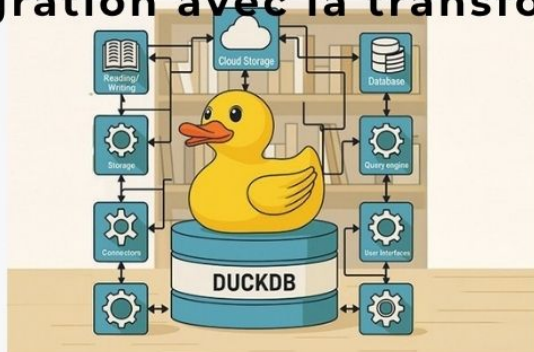
Après l'ingestion

DuckDB

- Stockage local des données brutes
- Source pour les étapes de transformation

Dagster

- Organisation du pipeline
- Intégration de l'ingestion dlt
- Enchaînement avec les étapes suivantes
- Intégration avec la transformation dbt



TESTS & RÉSULTAT

Validation du pipeline



Tests techniques

- Vérification du bon fonctionnement de DuckDB
- Utilisation d'un environnement isolé
- Vérification automatique avec pytest



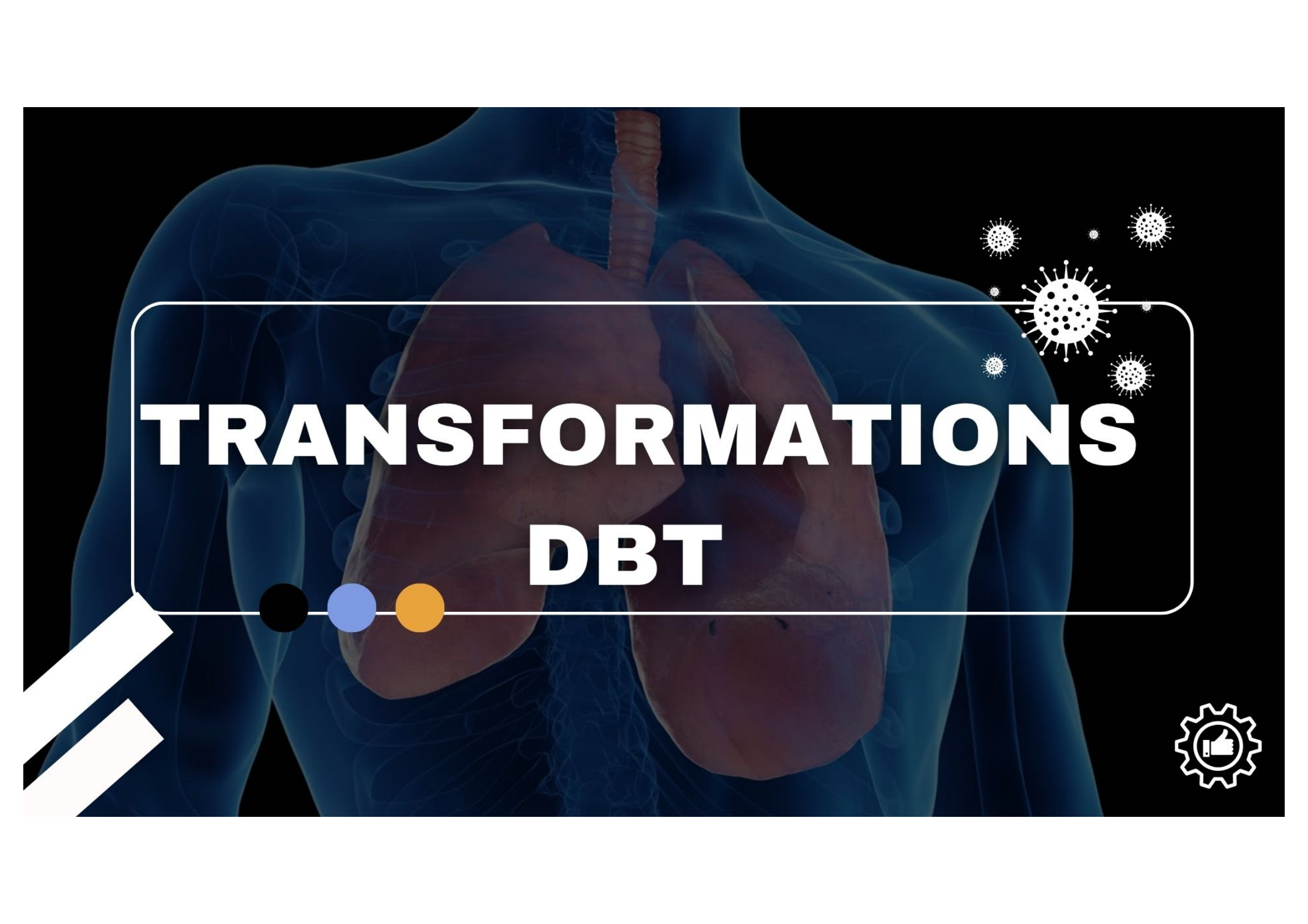
Flux global

Données brutes → dlt → DuckDB → Transformation

Résultat :

Données correctement ingérées, stockées et mises à disposition pour la transformation.





TRANSFORMATIONS

DBT

TÂCHE

- Récupérer les données brutes une fois chargées dans DuckDB
- Nettoyer et prétraiter les données
- Construire des modèles dbt
- Construire des tables de features transformées
- Préparer le dataset final pour le Machine Learning





ÉTAPE 1 : NETTOYAGE DE BASE

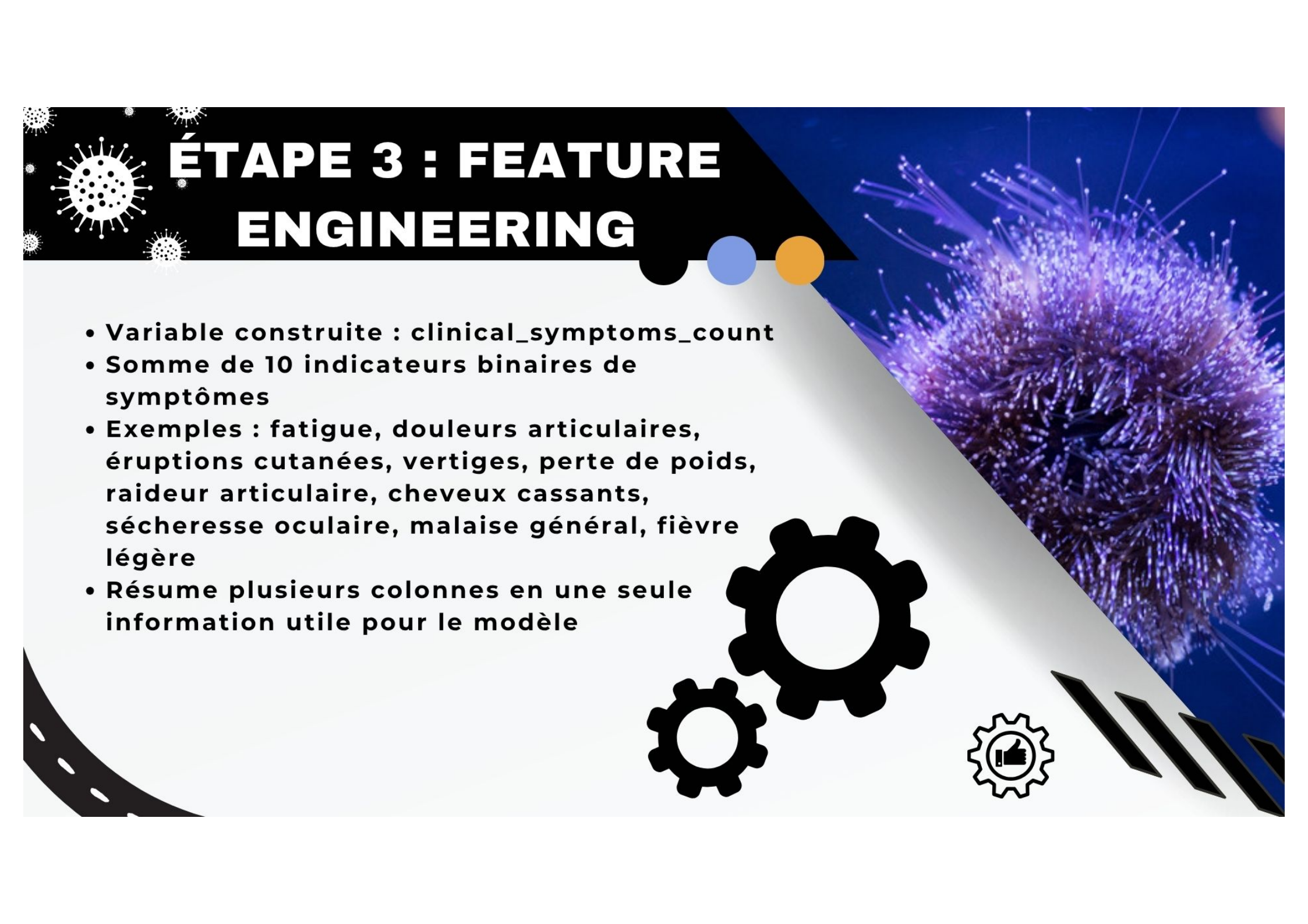
- Premier modèle dbt : partir des données brutes chargées dans DuckDB
- Normaliser le genre en minuscules
- Regrouper les libellés de diagnostic (texte libre, variantes) vers 6 catégories fixes
- Aucune ligne ni colonne supprimée à cette étape : version propre de la source
- Garder le patient_id à ce stade, pour pouvoir retrouver chaque ligne plus tard si besoin



ÉTAPE 2 : NETTOYAGE STATISTIQUE

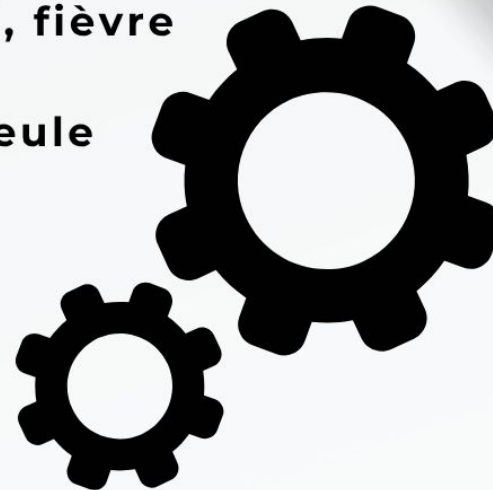
- Colonnes constantes : supprimer 2 colonnes qui n'ont qu'une seule valeur partout, donc inutiles
- Correction d'asymétrie : transformation logarithmique sur C4, distribution beaucoup plus normale
- Colonnes corrélées : supprimer 15 colonnes trop corrélées entre elles, pour éviter la redondance
- Suppressions fixées dans le code plutôt que recalculées à chaque fois, pour un résultat toujours identique
- Le notebook original détecte que la moitié des lignes sont des doublons stricts mais ne les supprime jamais : comportement reproduit fidèlement





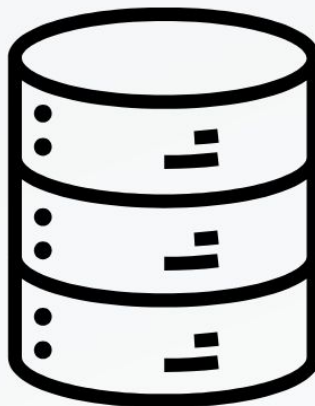
ÉTAPE 3 : FEATURE ENGINEERING

- Variable construite : `clinical_symptoms_count`
- Somme de 10 indicateurs binaires de symptômes
- Exemples : fatigue, douleurs articulaires, éruptions cutanées, vertiges, perte de poids, raideur articulaire, cheveux cassants, sécheresse oculaire, malaise général, fièvre légère
- Résume plusieurs colonnes en une seule information utile pour le modèle



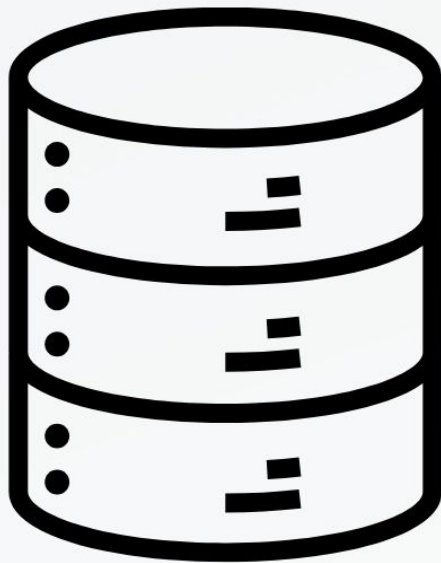
ÉTAPE 4 : DATASET FINAL ML

- Livrable final : dataset entièrement numérique, prêt pour l'entraînement
- Retirer le patient_id (ce n'est pas une feature)
- Encoder le genre et le diagnostic en chiffres
- Même mapping que le modèle déjà entraîné, pour rester compatible
- Dataset utilisé directement par l'équipe Machine Learning



RÉSULTAT

- 79 colonnes brutes → 60 colonnes finales
- 0 ligne perdue à travers tout le pipeline
- 4 modèles dbt (staging → intermediate → marts)



CONCLUSION

- Logique écrite en SQL, versionnée, et rejouable à l'identique — contrairement à un notebook manuel
- Rôle : transformer les données brutes en un dataset fiable et reproductible, prêt pour le Machine Learning





DATA QUALITY & DATA CONTRACTS





DATA QUALITY ENGINEER

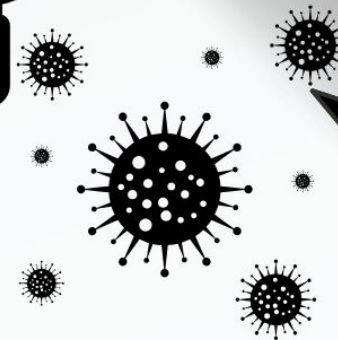
🎯 Ma mission

Garantir la fiabilité des données avant le Machine Learning

- Complétude
- Cohérence
- Conformité
- Contrôle à chaque étape du pipeline

Position dans le pipeline :

Transformations dbt → 🔍 Data Quality → 🤖 ML



LES DONNÉES DU PROJET



Dataset

13 812 patients

79 variables initiales

Données	Exemples
 Patients	Âge, sexe
 Bilan sanguin	Hémoglobine, globules blancs, plaquettes
 Marqueurs immunologiques	Présence / absence d'anticorps
 Symptômes	Fatigue, fièvre
 Diagnostic	6 catégories

6 diagnostics

Lupus • Polyarthrite rhumatoïde • Sjögren • Basedow • Orchite auto-immune • Normal

Position de mon rôle :

Données brutes → ... → dbt →  Data Quality → ML



DATA CONTRACT

Premier niveau de garantie

Data Contract — ml_patients_dataset

Le contrat définit :

-  60 colonnes
-  Types de données précis
-  Valeurs autorisées
-  Rejet immédiat en cas de non-conformité

Exemple

- genre → {0, 1}
-  Valeur inattendue
- → Construction échoue immédiatement

Objectif

- Détecter l'erreur à la frontière du Machine Learning
- dbt → Data Contract →  ML



4 TESTS AUTOMATISÉS

Second niveau de garantie
4 contrôles exécutés automatiquement à chaque reconstruction

Test	Ce qu'il vérifie	Type
1. Continuité des lignes	Nombre de patients conservé	● Bloquant
2. Cohérence des diagnostics	Diagnostics reconnus et pris en charge	● Bloquant
3. Taux de duplication	Référence 40-60 %	● Alerte
4. Volume de patients	Référence 500-800	● Alerte

⚙ **Automatisation**
Tests → CI/CD → GitHub Actions



CONCLUSION

Deux niveaux de garantie

① Data Contract

Fixe la forme de l'interface finale



② 4 tests automatisés

Vérifient la cohérence et la stabilité des données



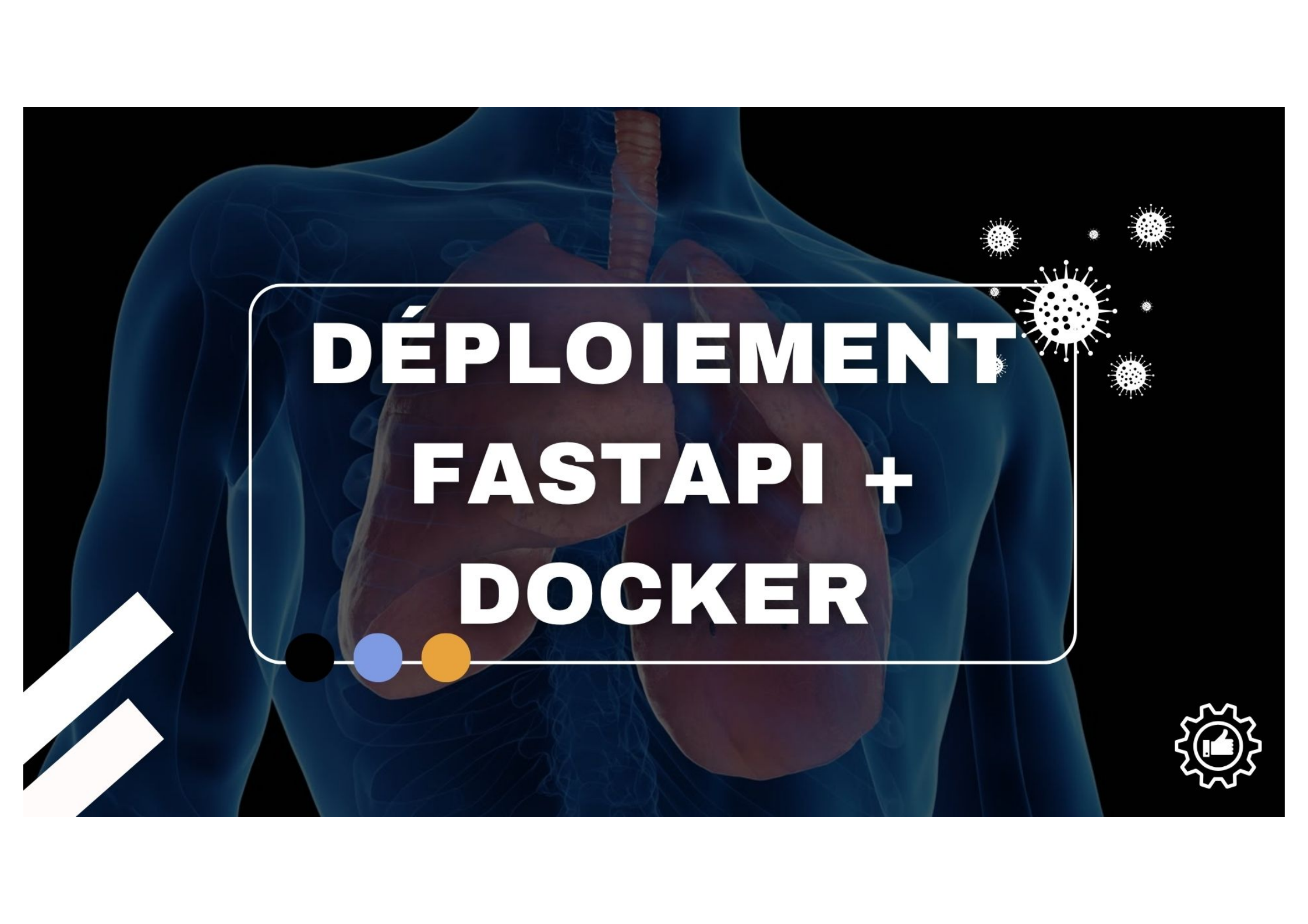
Machine Learning

Donnée vérifiée · contractualisée · surveillée



Automatiquement à chaque exécution du pipeline



The background of the slide features a blue-tinted illustration of a human torso, focusing on the chest and lungs. Several white, spiky virus-like icons are scattered in the upper right area. A white rounded rectangle frames the central text. At the bottom left of this rectangle, there are three colored circles (black, blue, orange) connected by a thin white line. In the bottom right corner of the slide, there is a white gear icon with a thumbs-up symbol inside it.

DÉPLOIEMENT FASTAPI + DOCKER

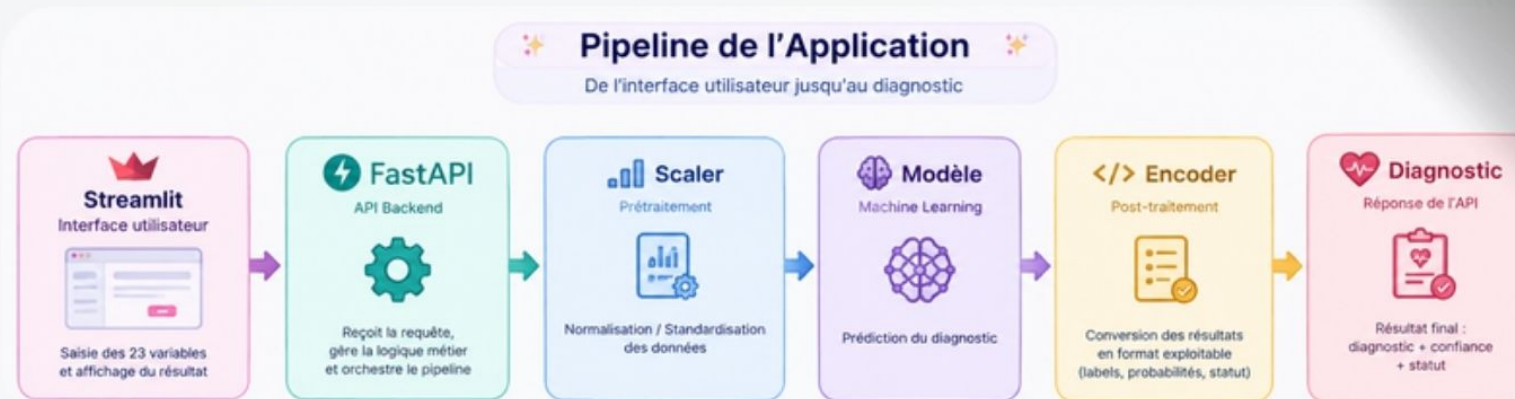
DEPLOYMENT LEAD

- **Intégration FastAPI**
- **Connexion au modèle entraîné**
- **Dockerisation**
- **Endpoints API : /predict et /health**
- **Intégration finale (frontend + monitoring)**



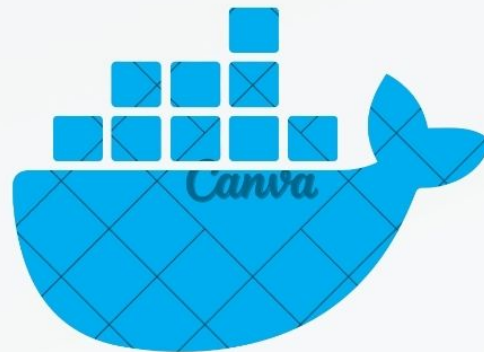
ARCHITECTURE DU SERVICE ET ENDPOINTS

- **Schéma** : Streamlit → FastAPI → Scaler → Modèle → Encoder → Diagnostic
- **POST /predict** : reçoit 23 features, retourne {diagnosis, status}
- **GET /health** : vérifie l'état de l'API et du modèle



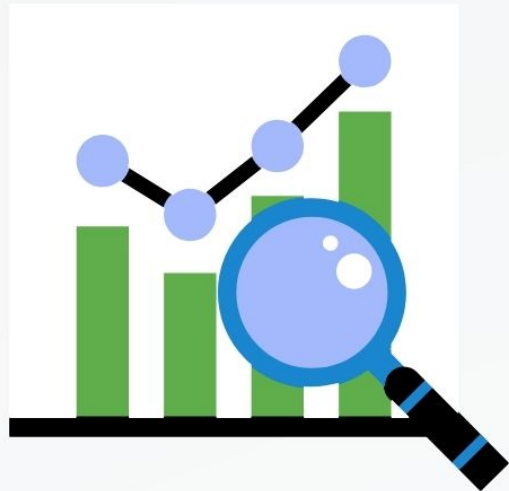
DOCKERISATIO

- **Dockerfile** : Python 3.11-slim, copie des artefacts, port 8000
- **docker-compose.yml** : lancement en une commande
- **Élimine le « ça marche sur ma machine »**



INTÉGRATION AVEC STREAMLIT ET MONITORING

- **Streamlit** : formulaire médical → appel API → affichage diagnostic + recommandations
- **Monitoring (Membre 7)** : middleware, /metrics, logs de prédiction, suivi de latence



ML & MLFLOW



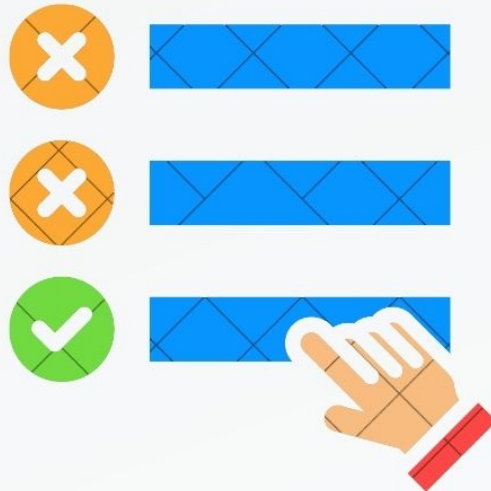
PRÉPARATION DES DONNÉES

- Ingestion depuis DuckDB vers Pandas.
- Split Train/Test (80/20).
- Normalisation des données numériques (StandardScaler).
- Équilibrage des classes avec SMOTEENN (uniquement sur le train set pour éviter le data leakage).



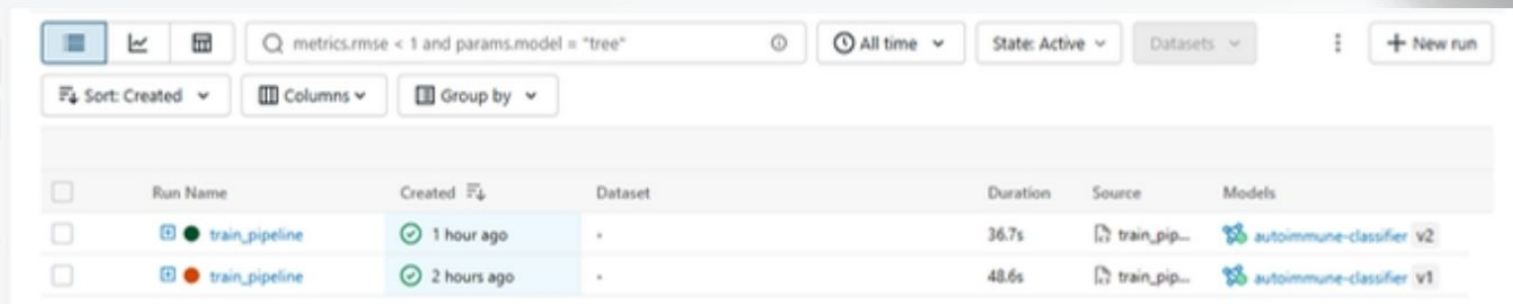
SÉLECTION DES FEATURES

- Utilisation de RFECV (Recursive Feature Elimination).
- Réduction drastique : passage de 79 à 23 features optimales.
- Liste figée pour garantir la cohérence avec l'API FastAPI.



ENTRAÎNEMENT & SÉLECTION DU MODÈLE

- **Deux modèles candidats** : Random Forest et SVC.
- **Évaluation multicritères** : Accuracy, Balanced Accuracy, F1-Score (weighted), ROC-AUC.
- **Sélection automatique du "Champion"** basée sur un score composite (moyenne du Balanced Accuracy et du F1).

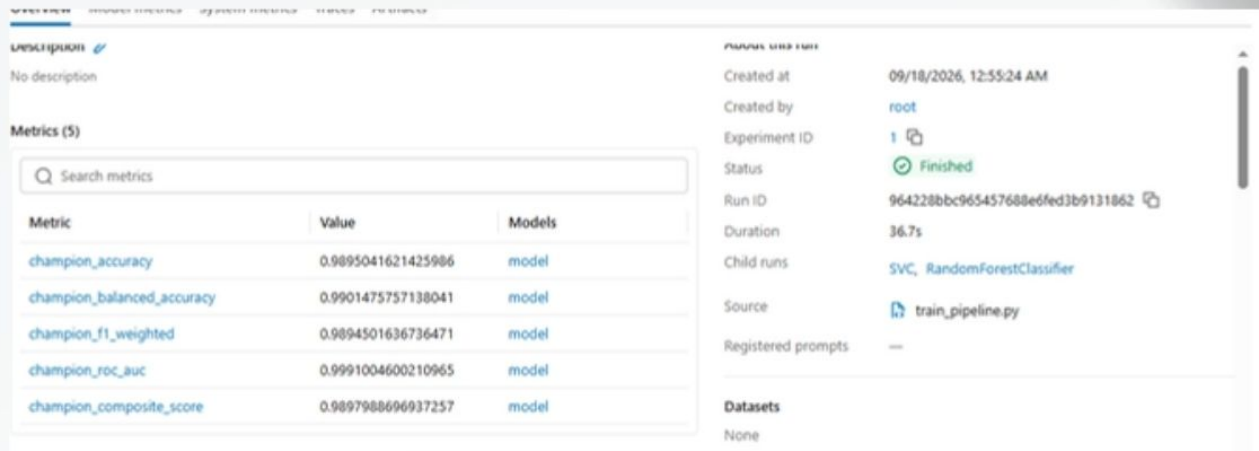


The screenshot shows a web interface for tracking machine learning runs. At the top, there is a search bar with the query "metrics.rmse < 1 and params.model = 'tree'", a filter for "All time", a "State: Active" dropdown, and a "Datasets" dropdown. Below this are controls for "Sort: Created", "Columns", and "Group by". The main table lists two runs, both named "train_pipeline". The first run was created "1 hour ago", has a duration of "36.7s", and is associated with the model "autoimmune-classifier v2". The second run was created "2 hours ago", has a duration of "48.6s", and is associated with the model "autoimmune-classifier v1".

☐	Run Name	Created ↕	Dataset	Duration	Source	Models
☐	 train_pipeline	✓ 1 hour ago	-	36.7s	train_pip...	 autoimmune-classifier v2
☐	 train_pipeline	✓ 2 hours ago	-	48.6s	train_pip...	 autoimmune-classifier v1

ORCHESTRATION & SUIVI MLFLOW

- **Script central** : train_pipeline.py.
- **MLflow Tracking** : Enregistrement des paramètres, métriques et artefacts (Scalers, Features, Encoders).
- **MLflow Model Registry** : Le modèle gagnant est enregistré avec l'alias "champion".
- **Génération des artefacts de production** : production_model.pkl et model_metadata.json.



The screenshot displays the MLflow Tracking interface for a specific run. On the left, the 'Metrics (5)' section shows a table of performance metrics. On the right, the 'Run parameters' section lists key metadata about the run, including its creation time, creator, status, and source code.

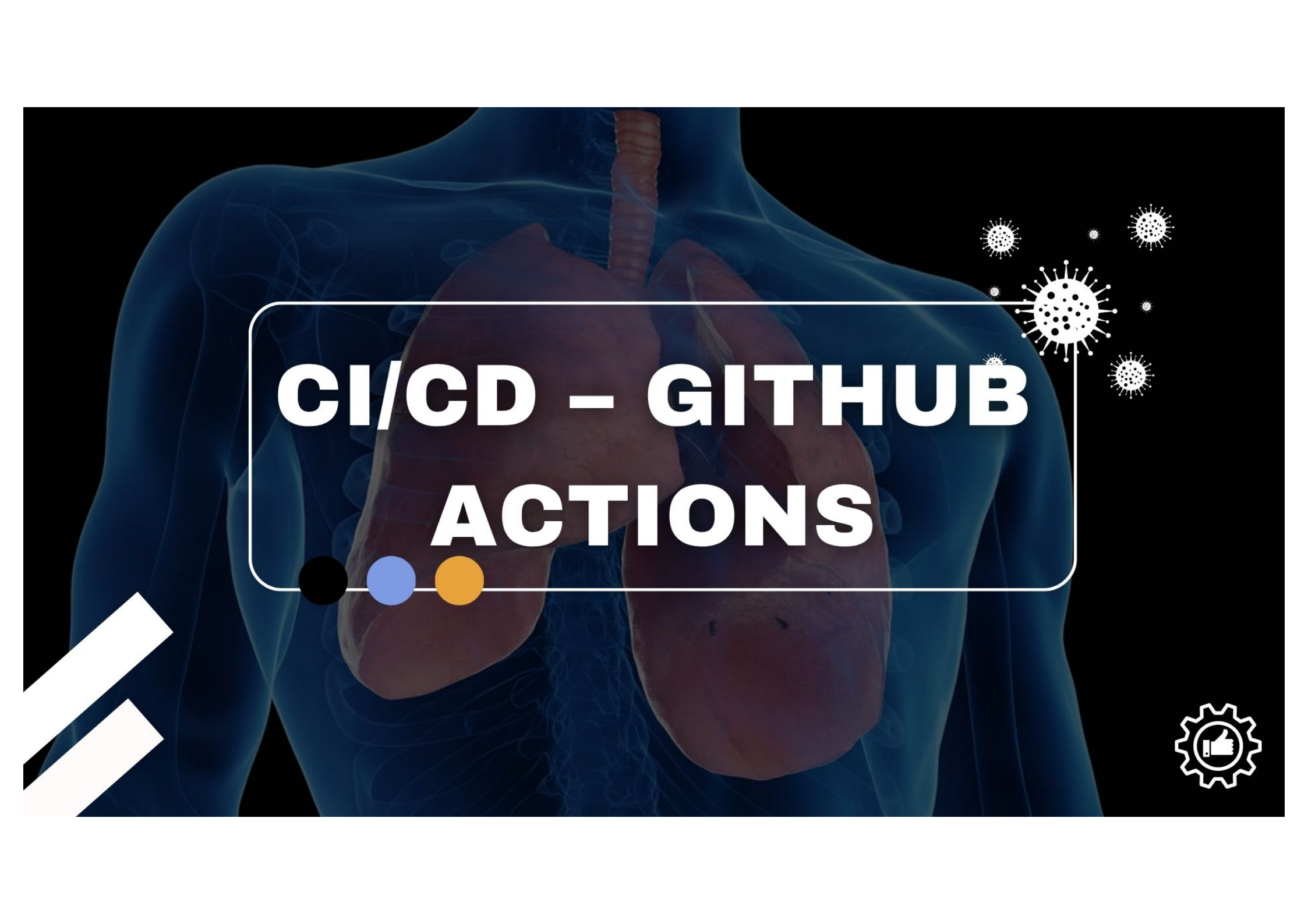
Metric	Value	Models
champion_accuracy	0.9895041621425986	model
champion_balanced_accuracy	0.9901475757138041	model
champion_f1_weighted	0.9894501636736471	model
champion_roc_auc	0.9991004600210965	model
champion_composite_score	0.9897988696937257	model

Run parameters	
Created at	09/18/2026, 12:55:24 AM
Created by	root
Experiment ID	1 🔗
Status	Finished
Run ID	964228bbc965457688edfed3b9131862 🔗
Duration	36.7s
Child runs	SVC , RandomForestClassifier
Source	train_pipeline.py
Registered prompts	—
Datasets	
None	

INTÉGRATION & DÉPLOIEMENT

- **L'API FastAPI** charge dynamiquement le modèle "champion" et son scaler.
- **Les 23 features** sont extraites automatiquement du modèle (`feature_names_in_`) pour éviter tout mismatch.
- **Prêt pour le monitoring en production.**



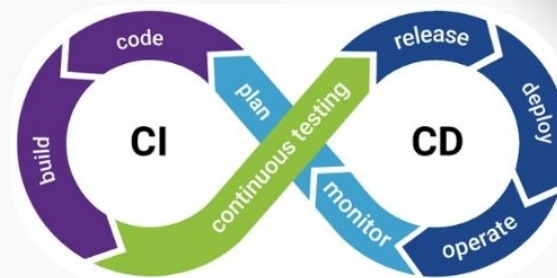


CI/CD – GITHUB ACTIONS



GITHUB ACTIONS & CI/CD

- * *DevOps / CI-CD*
- * *GitHub Actions*
- * *Automatisation du pipeline*
- * *Triggers :* Push | Pull Request
- * *Branches :* main | dev
- * *Code Quality :*
 - * Flake8 → Code
 - * Black → Formatage
 - * isort → Imports
- * *needs → dépendances entre jobs*



TESTS & VALIDATION

- *** Automated Validation***
- *** DataOps***
- *** DLT → Ingestion**
- *** DuckDB → Database**
- *** Artifacts → transfert entre jobs***
- *** Data Transformation & Quality***
- *** dbt**
- *** Data Quality Tests**
- *** Orchestration → Dagster***
- *** ML***
- *** Training**
- *** pytest → Backend & Monitoring**
- *** Objectif :*** chaîne de validation automatisée



DOCKER INTEGRATION & E2E

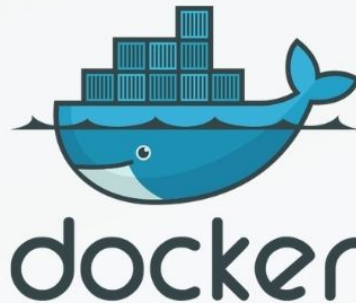
- *** * Docker Integration***

- docker compose build
- docker compose up -d
- *** * Health Check → /health***
- *** * E2E Testing***

- Frontend → :8501
- Backend → /health
- /features
- /predict
- Prediction Test

- Sample patient data
- Response validation

Objectif : * vérifier le fonctionnement des conteneurs

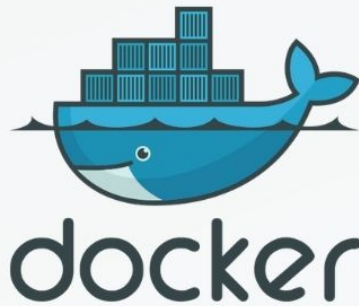


DOCKERHUB

- *** *Continuous Delivery***
- *** *Branch :* main**
- *** *Authentication***

- **DOCKERHUB_USERNAME**
- **DOCKERHUB_TOKEN**
- *** *5 Docker Images***

- **Backend**
- **Frontend**
- **Dagster**
- **Monitoring**
- **Dashboard**
- **Tag :* latest**
- *** *Automatic Docker Image Push***
- *** *Objectif* :* automatiser la publication**

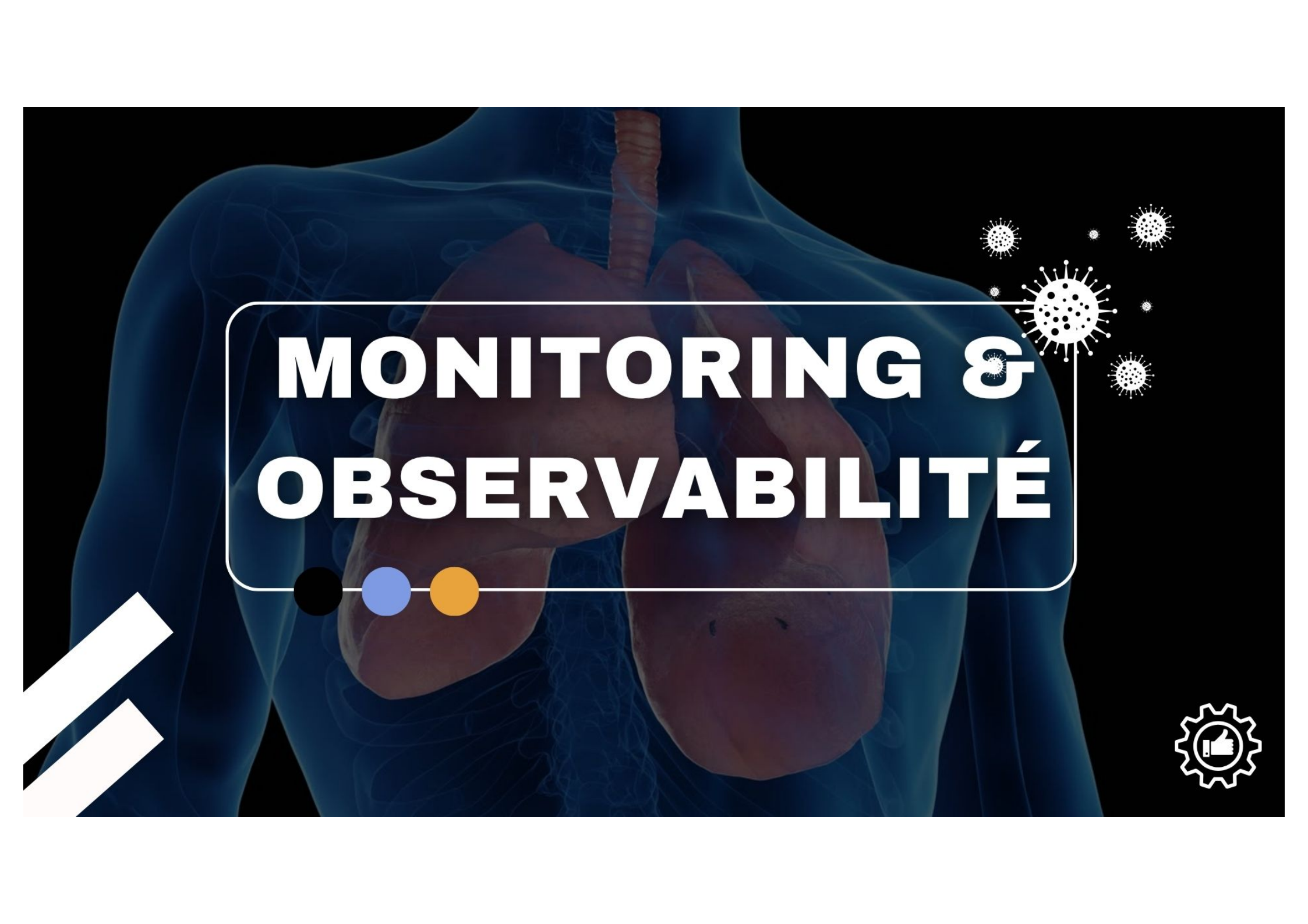


KOMODO

- *** Automated Deployment***
- *** Dependency :* docker-push → komodo-deploy**
- *** Authentication***

- **KOMODO_API_KEY**
- **KOMODO_API_SECRET**
- *** Komodo SDK → DeployStack***
- *** Stack :* autoimmune_detection_system**
- *** Final Pipeline :*
Quality → Tests → Docker → DockerHub → Komodo → Deployment***
- *** *Objectif* :* réduire les opérations manuelles et sécuriser la livraison**





MONITORING & OBSERVABILITÉ

TÂCHE

- Surveiller l'application après son déploiement
- Centraliser les logs applicatifs et les prédictions
- Mesurer les performances de l'API
- Vérifier la disponibilité du backend
- Détecter les changements dans les données de production
- Centraliser les indicateurs dans un dashboard



ÉTAPE 1 : LOGGING STRUCTURÉ*

- Mettre en place le logging dans **logging_config.py**
- Utiliser un format JSON structuré et exploitable automatiquement
- Enregistrer pour chaque prédiction :

* Features reçues

- Classe prédite
- Latence
- Statut de la requête
- Request ID
- Utiliser les logs pour le diagnostic, le dashboard et la détection de drift



ÉTAPE 2 : MONITORING DE L'API

- Utiliser un middleware FastAPI dans `middleware.py`
 - Intercepter automatiquement les requêtes avant et après traitement
 - Mesurer :
 - Nombre de requêtes
 - Nombre / taux d'erreurs
 - Latence des réponses
- Exposer les métriques via `/metrics`
-Format compatible avec Prometheus
-Suivre la latence avec `*p50, p95 et p99*`



ETAPE 3 : HEALTH MONITORING

- Vérifier périodiquement l'endpoint /health
- Effectuer un contrôle environ toutes les *15 secondes*
- Enregistrer les résultats dans health.log
- Suivre :
 - Statut du service
 - Temps de réponse
 - Historique des contrôles
 - Calculer le taux de disponibilité (*uptime*)
 - Vérifier que le backend répond réellement, et pas uniquement que le conteneur existe



ÉTAPE 4 : DÉTECTION DE DATA DRIFT*

- Comparer les données de référence aux données reçues en production
- Référence : * features du dataset ML final stocké dans DuckDB
- Production : * features enregistrées dans predictions.log
- Utiliser deux méthodes :
- PSI — Population Stability Index*
- KS — Kolmogorov-Smirnov*
- Générer automatiquement drift_report.json
- Seuils PSI :*
- $< 0,10 \rightarrow$ *STABLE*
- $0,10 - 0,25 \rightarrow$ *WATCH*
- $> 0,25 \rightarrow$ *DRIFT*



DASHBOARD STREAMLIT

- Centraliser les informations de monitoring dans une seule interface
- Visualiser :
 - Disponibilité du service
 - Nombre de prédictions
 - Taux d'erreur
 - Latence p50 / p95 / p99
 - Distribution des classes prédites
 - État du drift par feature
- Exécuter le monitoring et le dashboard dans des conteneurs Docker dédiés



Monitoring

Autoimmune Disease Prediction —
service observability

API base URL

http://ml-backend:8000

Monitoring

Autoimmune Disease Prediction —
service observability

API base URL

http://localhost:8000

Probe health now

Refresh metrics

AVAILABILITY

97.34 %

UP · 526 probes

PREDICTIONS SERVED

128

0 errors · 0.0% error rate

LATENCY P95

52.9 ms

p50 22.87 · p99 73.31 ms

FEATURE DRIFT

0 / 9 watch

over 104 samples ·

WATCH

LATENCY TREND (LAST 50)



PREDICTED CLASS DISTRIBUTION



Monitoring

Autoimmune Disease Prediction —
service observability

API base URL

`http://ml-backend:8000`

Monitoring

Autoimmune Disease Prediction —
service observability

API base URL

`http://localhost:8000`

Probe health now

Refresh metrics

PER-FEATURE DRIFT (PSI / KS)

feature	PSI	KS p-value	ref mean	current mean	status
PLT_Count	0.115800	0.781100	294699.395000	295276.275000	WATCH
MCH	0.111300	0.410000	29.218000	28.655000	WATCH
MBL_Level	0.107800	0.682500	1.278000	1.276000	WATCH
Hemoglobin	0.096400	0.436000	12.855000	12.662000	STABLE
C4	0.094700	0.492300	1.894000	1.893000	STABLE
Neutrophils	0.092100	0.204900	52.188000	52.694000	STABLE
Eosinophils	0.091400	0.473300	2.910000	3.044000	STABLE
Reticulocyte_Count	0.084000	0.387000	1.828000	1.778000	STABLE
MCHC	0.077300	0.648300	33.480000	32.999000	STABLE
WBC_Count	0.069400	0.759100	7953.156000	7828.929000	STABLE

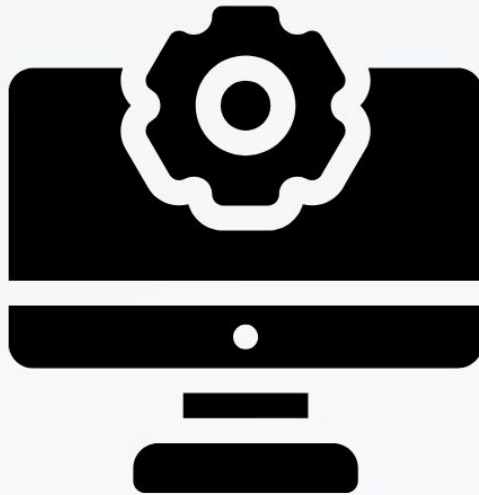
PSI thresholds — <0.10 stable · 0.10–0.25 watch · >0.25 drift. KS drift flagged when $p < 0.05$.

ARCHITECTURE DU MONITORING

- ***DuckDB*** → données de référence
- ***FastAPI /predict*** → predictions.log
- ***FastAPI /metrics*** → métriques API
- ***FastAPI /health*** → health.log
- ***Drift Detector*** → drift_report.json

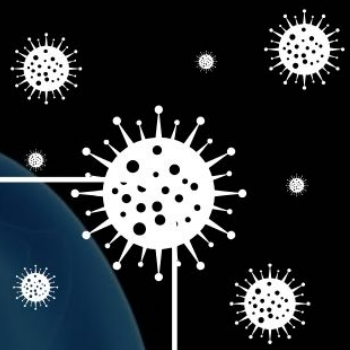
↓

- ***Dashboard Streamlit***





RÉSULTATS, LIMITES



LIMITE & CONCLUSION

- Surveiller le fonctionnement du système après déploiement
- Détecter les erreurs, problèmes de disponibilité et changements de distribution
- Limite actuelle : absence de *ground truth* en production
- Impossible de recalculer directement l'accuracy ou le F1-score sans labels réels
- Possibilité d'intégrer ces métriques si les vrais diagnostics deviennent disponibles

Résultat :

- * Monitoring centralisé et automatisé
- * Suivi de l'API, du service et des données
- * Pipeline MLOps observable après déploiement





**THANK'S FOR
WATCHING**

